

Maragal_8 - Run Instructions

Step-by-step procedure for reproducing the benchmark

1. Before starting

This document gives the procedure for reproducing the Maragal_8 benchmark from the supplied files. It is written so that a reader who is not already familiar with scientific Python or Slurm can follow the overall process. The exact University CSF login, partition and resource rules can change, so current University documentation must be followed for those institutional details.

The full calculation should be treated as an HPC job. The benchmark was designed around execution on the University CSF rather than as a normal desktop calculation.

2. Required files

File	Purpose
Maragal_8.py	Main benchmark.
plot_Maragal_8.py	Creates plots from generated results.
The three PDFs	Explain the experiment, run procedure and restrictions.

3. Access the CSF

A valid University account and permission to use the CSF are required. The current login procedure must be obtained from the University's CSF documentation. Once logged in, copy Maragal_8.py and plot_Maragal_8.py into a working directory.

4. Check Python

The program requires Python 3 together with NumPy, pandas and SciPy. Check the environment supplied by the CSF before submitting the job.

```
python --version python -c "import numpy, pandas, scipy; print('Scientific Python packages OK')"
```

If the import check fails, use the University's supported Anaconda/Conda or Python environment instructions. Do not assume that the login environment contains every required package.

5. What the main script does

Maragal_8.py locates the Matrix Market file, optionally downloads and extracts the SuiteSparse archive, loads the matrix as float64 CSR, constructs $b = A @ \text{ones}$, then runs PCG, LSQR and LSMR for every lambda. The default values are:

```
1e-6, 1e-5, 1e-4, 1e-3, 1e-2, 1e-1, 1.0
```

The default benchmark uses relative tolerance 1e-8, maximum 5000 iterations and three repeats per solver/lambda combination.

6. Obtain the matrix and run the benchmark

If the compute environment permits the script to download the dataset, the default route is:

```
python Maragal_8.py --download-data
```

The default data directory is suitesparse_data and the default output directory is real_data_pcg_output. If the dataset is already present, use:

```
python Maragal_8.py --data-dir suitesparse_data
```

The program prints the matrix dimensions and nonzero count, followed by progress for every lambda, solver and repeat.

7. Submit through Slurm

The full calculation should be submitted as a Slurm job. The dissertation workflow used a 24-hour wall-time allocation. Memory, CPU count, partition and account are user- and University-dependent and must be checked against the current CSF rules.

```
#!/bin/bash #SBATCH --job-name=maragal8 #SBATCH --time=24:00:00 #SBATCH --mem=<CURRENT_MEMORY_LIMIT>
#SBATCH --cpus-per-task=<CURRENT_CPU_COUNT> #SBATCH --output=maragal8_%A.out # Activate the required
Python environment. python Maragal_8.py --download-data
```

The placeholders are intentional. They must be replaced with values permitted by the current CSF configuration. Do not copy an old job number or assume that an old partition name remains valid.

The dissertation workflow also used Slurm array execution for benchmark workloads. If an array is used again, each task must have clearly separated output and its index must identify the case being run.

8. Monitor the calculation

Use the current University's Slurm documentation for monitoring commands. A common command is:

```
squeue -u $USER
```

The output log should show the dataset, matrix size, lambda sweep and solver progress. If a job ends unexpectedly, inspect its Slurm output before changing the code.

9. Locate the generated CSV files

A successful benchmark writes three CSV files inside `real_data_pcg_output`:

CSV	Purpose
Maragal_8_lambda_sweep_results.csv	Detailed results for every solver, lambda and repeat.
Maragal_8_lambda_sweep_summary.csv	Mean and standard-deviation summary.
Maragal_8_pcg_lsqr_comparison.csv	Mean PCG/LSMR timing comparison and time ratio.

These are generated outputs. They do not need to be permanently stored in the repository if the scripts remain available to recreate them.

10. Create the plots

Run the plotting script after the CSV files have been created. The exact input and output locations should follow the paths implemented in `plot_Maragal_8.py`.

```
python plot_Maragal_8.py
```

The script reads the benchmark CSV data and produces the PNG figures. If it reports a missing file, check that the benchmark has completed and that the plotting script is being run from the expected directory.

11. Copy results back

CSV files, PNG figures and Slurm logs can be copied from the CSF to a local machine using the University's approved transfer method. They are outputs, not source material, so they can be kept separately from the repository.

12. What counts as a successful reproduction?

The important check is not that every runtime is numerically identical. Hardware load and software versions can change timings. A successful reproduction should instead use the same Maragal_8 matrix, construct b as $A @ \text{ones}$, test the same lambda sweep, retain the supplied tolerance and iteration limit, and use the same PCG matrix-free Jacobi-preconditioned formulation. The resulting CSVs should contain the intended solver/lambda/repeat

combinations.